

СИСТЕМНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

БЛОК 2

Ляхманов Д.А.

к.т.н., доцент

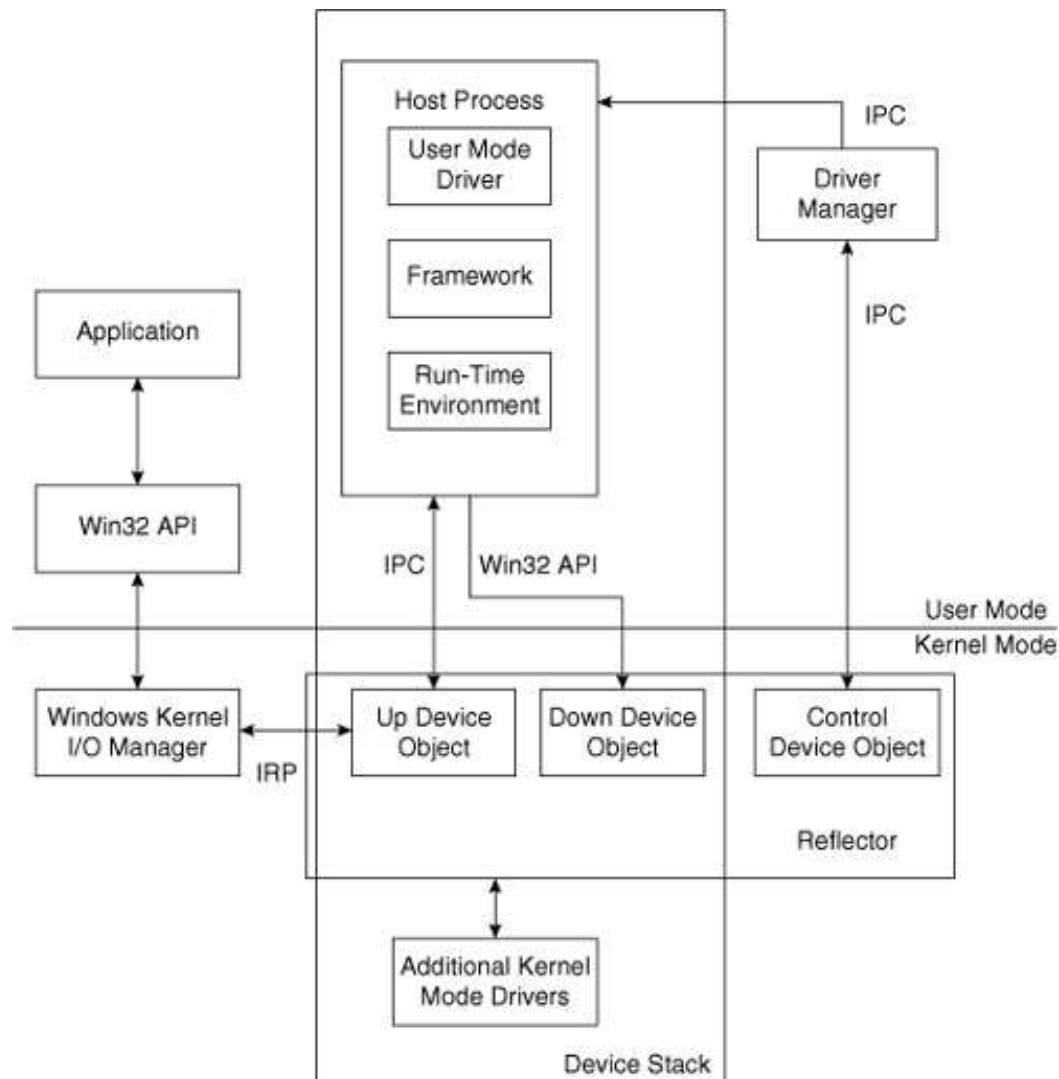
СТРУКТУРА UMDF-ДРАЙВЕРА

РЕАЛИЗАЦИЯ PNP-ДЕРЕВА

Причины использования технологии COM:

- многие программисты уже знакомы с этой технологией;
- интерфейсы COM позволяют логическое группирование функций, что способствует пониманию и передвижению по интерфейсу DDI с легкостью;
- существуют многочисленные инструменты, включая библиотеку ATL, поддерживающие приложения и объекты на основе COM

ИНФРАСТРУКТУРА UMDF-ДРАЙВЕРА



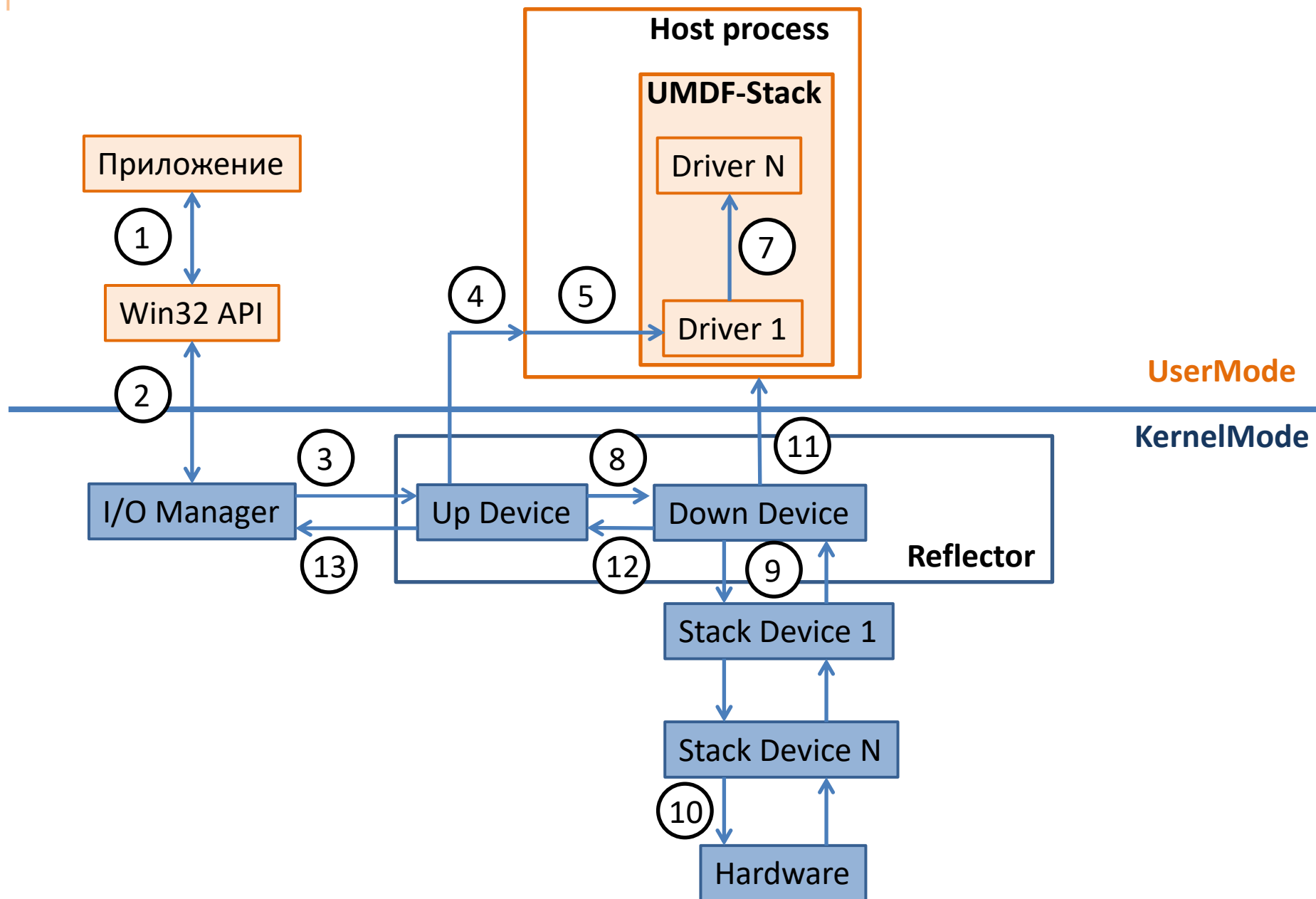
ИНФРАСТРУКТУРА UMDF-ДРАЙВЕРА

- **Хост-процесс** — содержит компоненты, составляющие часть стека устройства, исполняющегося в пользовательском режиме, и служит посредником во взаимодействии между этими компонентами и отражателем. Каждый стек UMDF имеет собственный хост-процесс
- **Reflector** — это драйвер режима ядра, который поддерживает все драйверы UMDF в системе. Он управляет взаимодействием между компонентами режима ядра, такими как стеки устройств режима ядра, и хост-процессами UMDF
- **Менеджер драйверов (Driver Manager)** — это сервис Windows, который управляется всеми хост-процессами UMDF в системе. Он создает хост-процесс, отслеживает состояние во время исполнения хост-процессов и останавливает хост-процессы, когда они больше не нужны
- **Объект устройства Up (Up Device Object)** — объект, который устанавливается вверху стека устройства режима ядра для устройства. Он принимает запросы от менеджера ввода/вывода, передает их хост-процессу UMDF для обработки и после их завершения возвращает менеджеру ввода/вывода

ИНФРАСТРУКТУРА UMDF-ДРАЙВЕРА

- **Объект устройства Down (Down Device Object)** — объект, который не установлен в стеке устройства режима ядра. Он принимает запросы ввода/вывода от хост-процесса UMDF после их обработки драйвером и передает эти запросы нижележащему стеку устройства, который осуществляет взаимодействие с устройством.

ПОРЯДОК ОБРАБОТКИ ЗАПРОСОВ



ПОРЯДОК ОБРАБОТКИ ЗАПРОСОВ

1. Приложение вызывает функцию Windows
2. Windows вызывает соответствующую процедуру ввода/вывода режима ядра, которая передает запрос менеджеру ввода/вывода
3. Менеджер ввода/вывода создает пакет IRP и посылает его объекту устройства Up рефлексора
4. Up-устройство посылает запрос хост-процессу драйвера. Хост-процесс создает пакет IRP пользовательского режима для представления пакета IRP в части пользовательского режима стека устройства
5. Хост-процесс передает пакет IRP пользовательского режима верхнему экземпляру инфраструктуры в стеке устройства
6. Инфраструктура использует данные с пакета IRP пользовательского режима для создания IPC-запроса и передает его соответствующему объекту обратного вызова драйвера
7. Драйвер обрабатывает запрос ввода/вывода и посылает его следующему драйверу
8. Хост-процесс посылает обработанный IRP-пакет объекту устройства Down рефлексора, который создает второй пакет IRP

ПОРЯДОК ОБРАБОТКИ ЗАПРОСОВ

9. Объект устройства Down передает запрос нижнему стеку устройства, который обрабатывает запрос и осуществляет необходимое взаимодействие с устройством
10. Драйвер в нижнем стеке устройства завершает второй пакет IRP
11. Объект устройства Down возвращает выполненный запрос ввода/вывода хост-процессу драйвера и осуществляет обратный вызов для завершения запроса
12. Инфраструктура передает завершенный запрос ввода/вывода объекту устройства Up
13. Объект устройства Up передает запрос обратно менеджеру ввода/вывода, который возвращает результаты исполнения запроса приложению

КОМПОНЕНТЫ ХОСТ-ПРОЦЕССА

- **Стек UMDF**

- Состоит из одного или нескольких функциональных драйверов или драйверов фильтра
- Имеет свои объекты устройств и пакеты IRP, связанные с лежащими ниже структурами данных
- Обрабатывает запросы ввода/вывода (аналогично стеку режима ядра)

- **Один или несколько экземпляров инфраструктуры**

- Один экземпляр для каждого драйвера UMDF в стеке
- Представляет собой библиотеку, которая поддерживает объектную модель UMDF
- Служит посредником во взаимодействии между драйверами и хост-процессом

ПРЕИМУЩЕСТВА И НЕДОСТАТКИ UMDF

Преимущества:

- Повышенная стабильность системы
- Пониженная угроза безопасности
- Возможность использования Windows API
- Упрощенная отладка в пользовательском режиме
- Использование возможностей C++ или C
- Производительность на уровне драйверов режима ядра

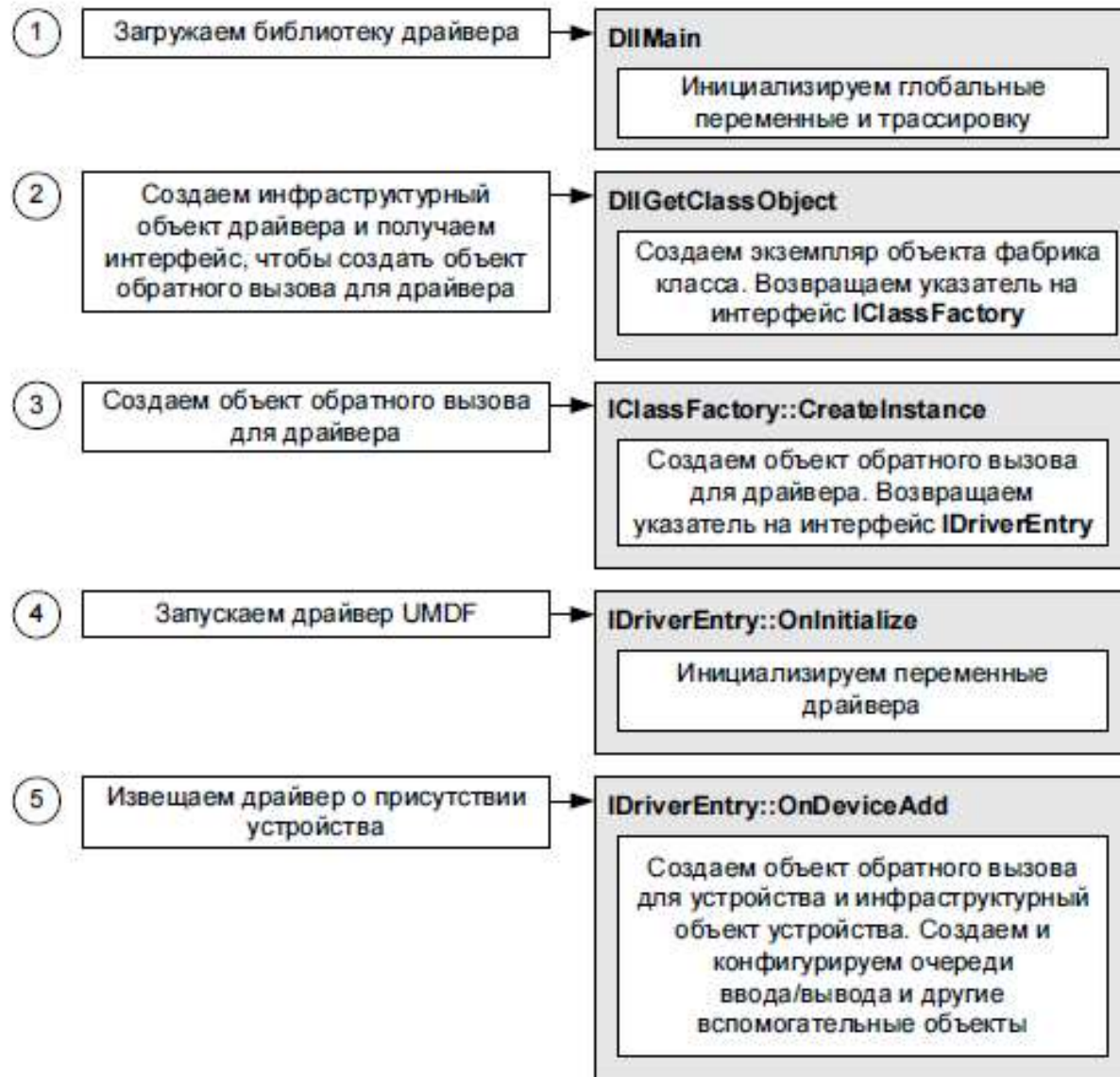
Недостатки:

- невозможность прямого доступа к аппаратной части, включая обработку прерываний
- нельзя реализовать бесперебойные временные циклы
- отсутствует прямой доступ к структурам данных или памяти ядра

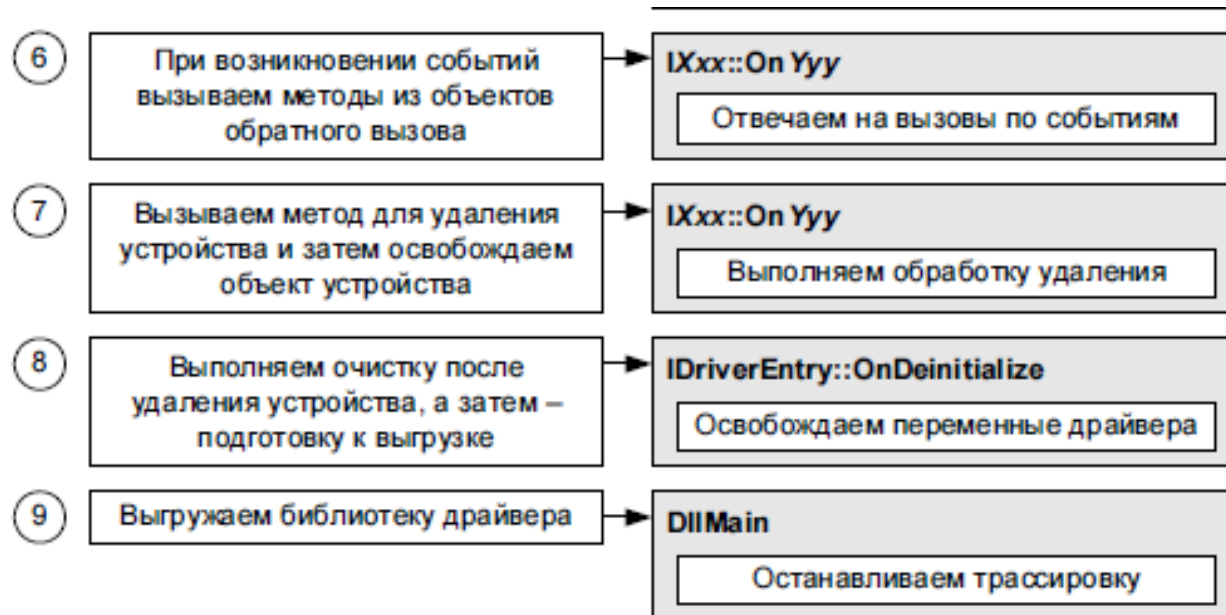
СОСТАВ UMDF-ДРАЙВЕРА

- реализовать функцию DllMain в качестве точки входа драйвера
- реализовать и экспортировать по имени функцию DllGetClassObject
- реализовать интерфейс IClassFactory для создания объекта драйвера
- реализовать объект обратного вызова драйвера, предоставляющий интерфейс IDriverEntry
- реализовать объект обратного вызова драйвера, предоставляющий интерфейсы обратного вызова для объекта устройства

ПОСЛЕДОВАТЕЛЬНОСТЬ ИНИЦИАЛИЗАЦИИ UMDF-ДРАЙВЕРА



ПОСЛЕДОВАТЕЛЬНОСТЬ ИНИЦИАЛИЗАЦИИ UMDF-ДРАЙВЕРА



СТРУКТУРА UMDF-ДРАЙВЕРА

ОБЯЗАТЕЛЬНЫЕ ФУНКЦИИ

Функция завершения драйвера вызывается, когда драйвер выгружается из оперативной памяти (напр. при отключении устройства):

```
BOOL WINAPI DllMain(  
    _In_ HINSTANCE hinstDLL,  
    _In_ DWORD fdwReason,  
    _In_ LPVOID lpvReserved )  
{ ... }
```

hinstDLL – описатель dll

fdwReason – флаг вызова функции (DLL_PROCESS_ATTACH,
DLL_PROCESS_DETACH, DLL_THREAD_ATTACH, DLL_THREAD_DETACH)

lpvReserved – зарегистрированный параметр

ОБЯЗАТЕЛЬНЫЕ ФУНКЦИИ

Функция, возвращающая указатель реализацию интерфейса IClassFactory:

```
HRESULT DllGetClassObject(  
    REFCLSID rclsid,  
    REFIID riid,  
    LPVOID *ppv)  
    {...}
```

`rclsid` – CLSID реализующего класса

`riid` – IID запрашиваемого интерфейса (IID интерфейса IClassFactory)

`ppv` – указатель на объект, реализующий интерфейс IClassFactory

БАЗОВЫЙ ИНТЕРФЕЙС ДРАЙВЕРА

Интерфейс, реализующий базовый функционал UMDF-драйвера (инициализацию, обработку добавления устройства, удаление драйвера из стека):

```
interface IDriverEntry
{
    HRESULT OnInitialize( IWDFDriver *pWdfDriver );
    void OnDeinitialize( IWDFDriver *pWdfDriver );
    HRESULT OnDeviceAdd( IWDFDriver *pWdfDriver,
        IWDFDeviceInitialize *pWdfDeviceInit );
};
```

ИНТЕРФЕЙС ОБЪЕКТА ДРАЙВЕРА

Интерфейс, реализующий базовый функционал UMDF-драйвера (инициализацию, обработку добавления устройства, удаление драйвера из стека):

```
interface IWDFDriver
{

HRESULT CreateDevice(
    IWDFDeviceInitialize *pDeviceInit,
    IUnknown *pCallbackInterface,
    IWDFDevice **ppDevice);

HRESULT CreatePreallocatedWdfMemory(
    BYTE *pBuff,
    SIZE_T BufferSize,
    IUnknown *pCallbackInterface,
    IWDFObject *pParentObject,
    IWDFMemory **ppWdfMemory);
```

ИНТЕРФЕЙС ОБЪЕКТА ДРАЙВЕРА

```
HRESULT CreateWdfMemory(  
    SIZE_T BufferSize,  
    IUnknown *pCallbackInterface,  
    IWDFObject *pParentObject,  
    IWDFMemory **ppWdfMemory);  
  
HRESULT CreateWdfObject(  
    IUnknown *pCallbackInterface,  
    IWDFObject *pParentObject,  
    IWDFObject **ppWdfObject);  
  
BOOL IsVersionAvailable(  
    UMDF_VERSION_DATA *pMinimumVersion );  
  
HRESULT RetrieveVersionString(  
    PWSTR pVersion,  
    DWORD *pdwVersionLength );  
};
```

СПАСИБО ЗА ВНИМАНИЕ